Recibe una cálida:

¡Bienvenida!

Te estábamos esperando 😁

alkemy

Clase grabada

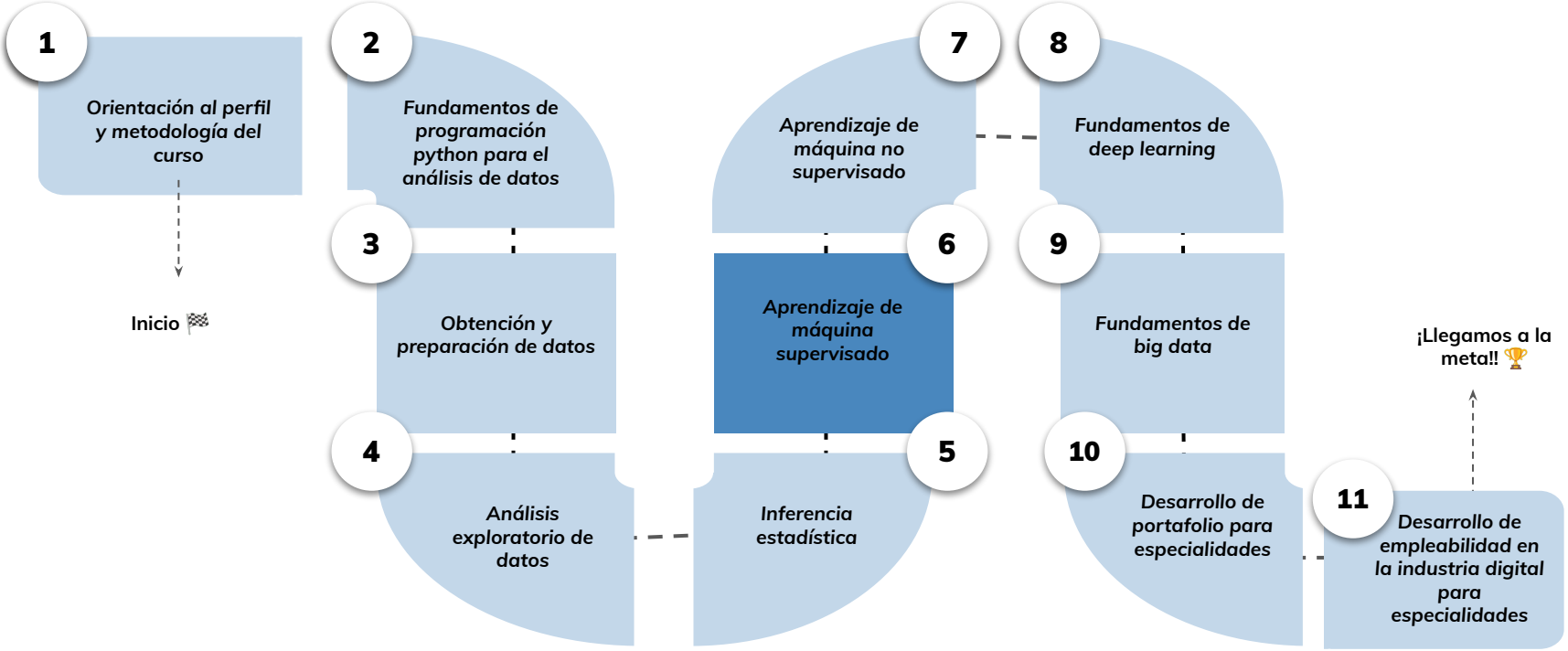
Algoritmos de clasificación- Parte II

Aprendizaje Esperado 5: Elaborar un modelo predictivo aplicando algoritmos de clasificación para resolver un problema dado de acuerdo a las buenas prácticas de la industria.

alkemy

Hoja de ruta

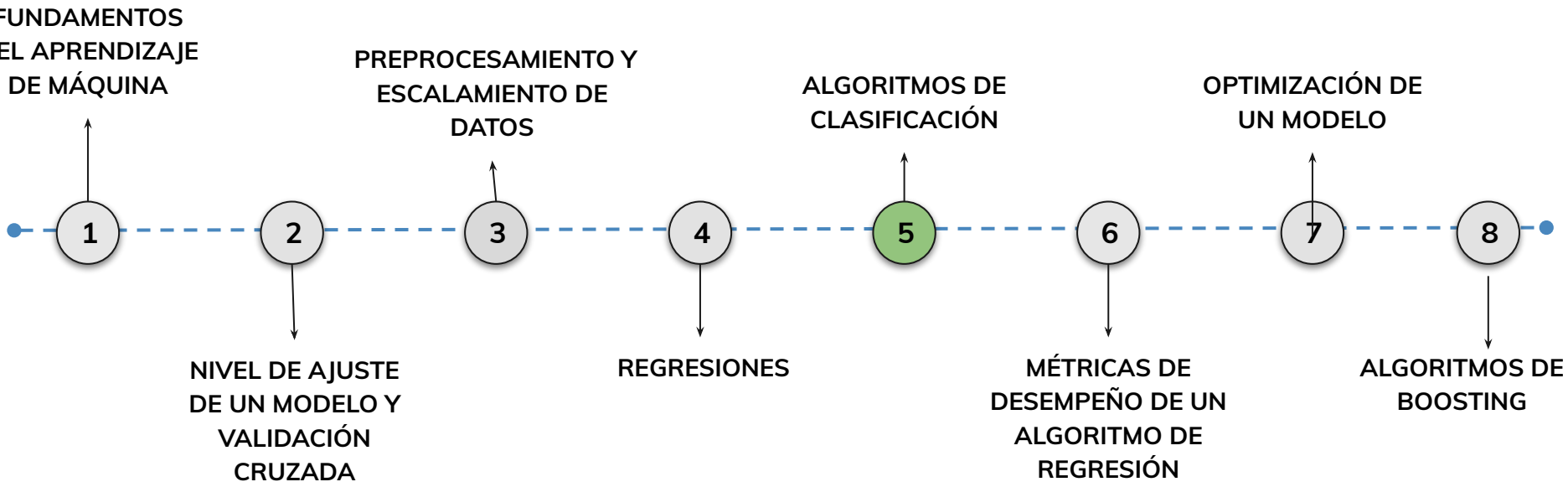
¿Cuáles skills conforman el programa? **Fundamentos de Ciencia de Datos**



alkemy

Roadmap de lecciones

*¿Cuáles **lecciones** estaremos estudiando en este módulo?*



alkemy

Learning Path

¿Cuáles temas trabajaremos hoy?

5.

Optimización y evaluación de modelos de clasificación

profundizaremos en el uso práctico de K-NN y cómo ajustarlo para mejorar su desempeño.

- **Implementación avanzada de K-NN**
  - Ajuste de hiperparámetros (K, métricas, pesos)
  - Visualización de fronteras de decisión
- **Validación y métricas**
  - Validación cruzada y AUC-ROC
  - Ajuste de umbral de decisión

alkemy

Objetivos de aprendizaje

¿Qué aprenderás?

- Implementar el algoritmo K-NN para resolver problemas reales de clasificación.
- Evaluar modelos de clasificación utilizando métricas específicas y visualizaciones.
- Optimizar hiperparámetros en K-NN para mejorar el rendimiento del modelo.
- Aplicar K-NN a tareas de regresión y comparar con otros algoritmos.
- Incorporar buenas prácticas para escalar K-NN a producción.

Repaso clase anterior

¿Quedó alguna duda?

En la clase anterior trabajamos 📚:

→ *Diferencias entre clasificación binaria y multiclase.*

→ *Implementación básica de algoritmos de clasificación con scikit-learn.*

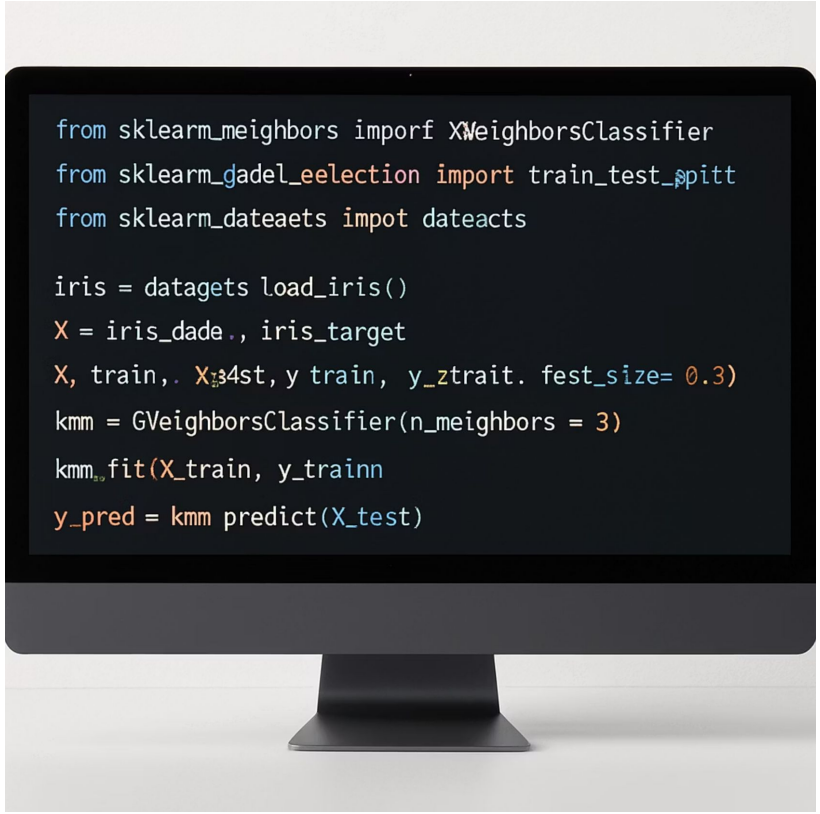
→ *Métricas de evaluación para problemas de clasificación.*

→ *Uso de la matriz de confusión y la curva ROC.*

Algoritmos de clasificación

Implementación en Python con la librería Scikit-Learn

La librería Scikit-Learn es una de las más utilizadas para implementar modelos de aprendizaje automático en Python. Esta herramienta ofrece un módulo específico para el algoritmo K-Nearest Neighbors (K-NN), facilitando todo el proceso de clasificación, desde el preprocesamiento de los datos hasta la evaluación del modelo.



Proceso de Implementación

Preparación de Datos

Asegurarse de que los datos estén correctamente estructurados, sin valores faltantes y con las variables categóricas codificadas de manera adecuada.

Escalamiento

Aplicar un escalamiento a las variables numéricas para evitar que la diferencia de magnitudes afecte el cálculo de la distancia.

División de Datos

Dividir los datos en un conjunto de entrenamiento y un conjunto de prueba utilizando la función train_test_split.

Definición del Modelo

Implementar el modelo K-NN utilizando la clase KNeighborsClassifier, especificando el número de vecinos y la métrica de distancia.

Evaluación

Evaluar el desempeño del modelo utilizando métricas como la exactitud, la matriz de confusión y la puntuación F1.

Ejemplo Completo de Implementación

```
import numpy as npimport matplotlib.pyplot as pltfrom sklearn.neighbors import KNeighborsClassifierfrom sklearn.model_selection import train_test_splitfrom sklearn.preprocessing import StandardScalerfrom sklearn.metrics import accuracy_score, confusion_matrix, classification_report# Datos de ejemplo (altura, peso, deporte)X = np.array([[180, 75], [165, 65], [170, 70], [160, 60],  [175, 72], [162, 63], [178, 74], [168, 68], [182, 78], [163, 64], [172, 71], [158, 59]])y = np.array(['Fútbol', 'Baloncesto', 'Fútbol', 'Baloncesto', 'Fútbol', 'Baloncesto', 'Fútbol', 'Baloncesto', 'Fútbol', 'Baloncesto', 'Fútbol', 'Baloncesto'])# Dividir datos en entrenamiento y pruebaX_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)# Escalar los datosscaler = StandardScaler()X_train_scaled = scaler.fit_transform(X_train)X_test_scaled = scaler.transform(X_test)# Crear y entrenar el modelo K-NNk = 3 # Número de vecinosknn = KNeighborsClassifier(n_neighbors=k)knn.fit(X_train_scaled, y_train)# Realizar prediccionesy_pred = knn.predict(X_test_scaled)# Evaluar el modeloaccuracy = accuracy_score(y_test, y_pred)conf_matrix = confusion_matrix(y_test, y_pred)class_report = classification_report(y_test, y_pred)print(f"Exactitud: {accuracy:.2f}")print("\nMatriz de Confusión:")print(conf_matrix)print("\nInforme de Clasificación:")print(class_report)
```

Salida del Ejemplo

```
Exactitud: 0.75Matriz de Confusión:[[2 0] [1 1]]Informe de Clasificación:              precision    recall
f1-score   support  Baloncesto       0.67      1.00      0.80         2      Fútbol       1.00      0.50
0.67         2    accuracy                           0.75         4   macro avg       0.83      0.75      0.73
4weighted avg       0.83      0.75      0.73         4
```

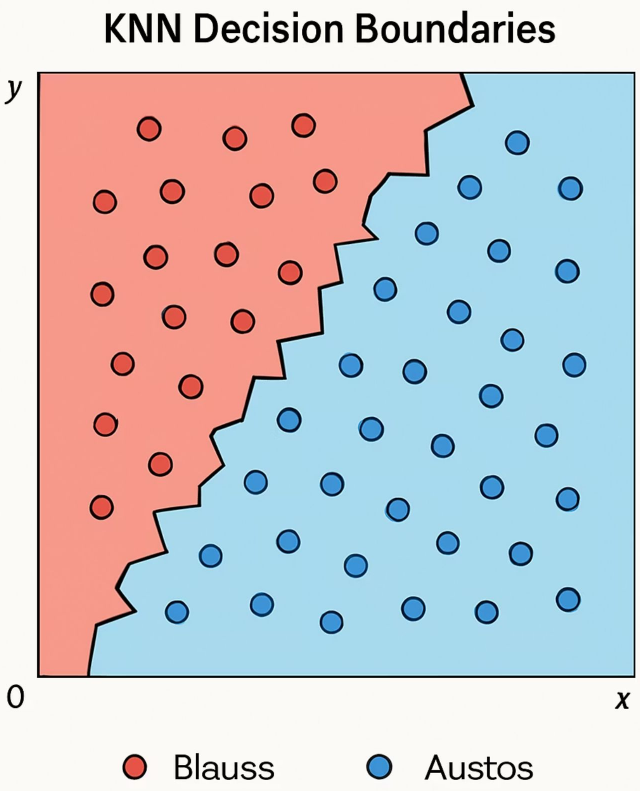
Este flujo completo permite implementar y evaluar un clasificador K-NN, garantizando un desempeño adecuado y facilitando la toma de decisiones en tareas de clasificación.

alkemy

Visualización de Fronteras de Decisión

Una forma útil de entender el comportamiento de K-NN es visualizar las fronteras de decisión que genera el algoritmo:

```
import numpy as npimport matplotlib.pyplot as pltfrom sklearn.neighbors import KNeighborsClassifierfrom sklearn.preprocessing import StandardScalerfrom matplotlib.colors import ListedColormap# Datos de ejemploX = np.array([[180, 75], [165, 65], [170, 70], [160, 60],  [175, 72], [162, 63], [178, 74], [168, 68]])y = np.array(['Fútbol', 'Baloncesto', 'Fútbol', 'Baloncesto', 'Fútbol', 'Baloncesto', 'Fútbol', 'Baloncesto'])# Escalar los datosscaler = StandardScaler()X_scaled = scaler.fit_transform(X)# Crear y entrenar el modelo K-NNk = 3knn = KNeighborsClassifier(n_neighbors=k)knn.fit(X_scaled, y)# Crear malla para visualizar fronteras de decisiónh = 0.02 # Tamaño del paso en la mallax_min, x_max = X_scaled[:, 0].min() - 1, X_scaled[:, 0].max() + 1y_min, y_max = X_scaled[:, 1].min() - 1, X_scaled[:, 1].max() + 1xx, yy = np.meshgrid(np.arange(x_min, x_max, h), np.arange(y_min, y_max, h))# Predecir clase para cada punto en la mallaZ = knn.predict(np.c_[xx.ravel(), yy.ravel()])Z = np.array([1 if z == 'Fútbol' else 0 for z in Z])Z = Z.reshape(xx.shape)# Visualizar resultadoplt.figure(figsize=(10, 8))plt.contourf(xx, yy, Z, alpha=0.8, cmap=ListedColormap(['#FFAAAA', '#AAAAFF']))plt.scatter(X_scaled[:, 0], X_scaled[:, 1], c=[1 if y_i == 'Fútbol' else 0 for y_i in y],  cmap=ListedColormap(['#FF0000', '#0000FF']), edgecolor='k', s=100)plt.title(f'Fronteras de Decisión con K-NN (k={k})')plt.xlabel('Altura (escalada)')plt.ylabel('Peso (escalado)')plt.show()
```



Resultado de la Visualización

Esta visualización muestra cómo K-NN divide el espacio de características en regiones de decisión. Cada color representa una clase diferente, y los puntos son los datos de entrenamiento.

Las fronteras de decisión ilustran cómo el algoritmo clasificará nuevas observaciones según su posición en el espacio.

Optimización de Hiperparámetros

Además de seleccionar el valor de K, existen otros hiperparámetros que pueden ajustarse para mejorar el rendimiento de K-NN:

- Métrica de distancia (euclidean, manhattan, minkowski, etc.)
- Ponderación de los vecinos (uniform, distance)
- Algoritmo para encontrar vecinos (auto, ball_tree, kd_tree, brute)

Scikit-Learn ofrece herramientas para la búsqueda automática de hiperparámetros:

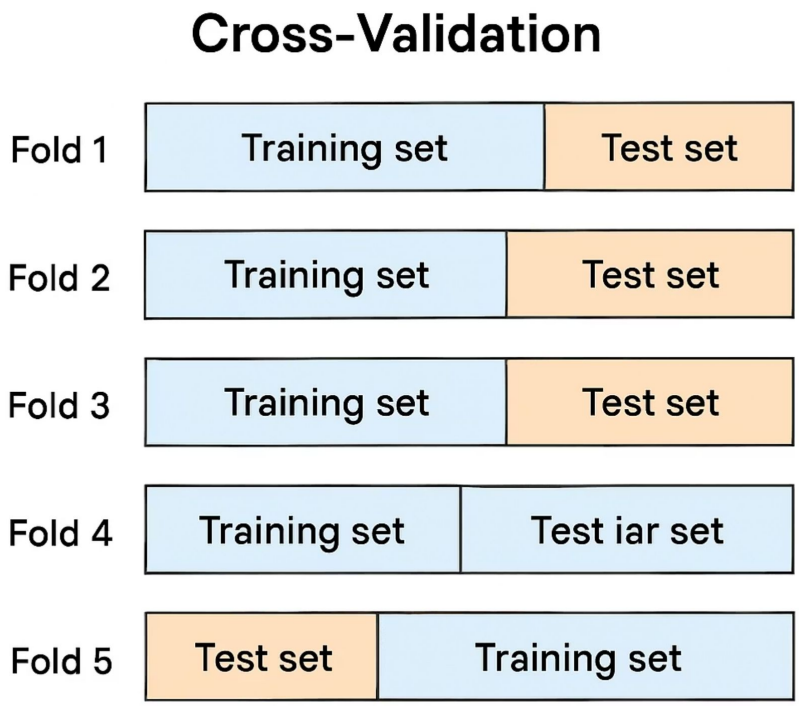
```
from sklearn.model_selection import GridSearchCV# Definir parámetros a probarparam_grid = { 'n_neighbors': range(1, 10), 'weights': ['uniform', 'distance'], 'metric': ['euclidean', 'manhattan']}# Crear modelo baseknn = KNeighborsClassifier()# Realizar búsqueda de cuadrículagrid_search = GridSearchCV(knn, param_grid, cv=5)grid_search.fit(X_train_scaled, y_train)# Obtener mejores parámetrosprint("Mejores parámetros:", grid_search.best_params_)
```

alkemy

Validación Cruzada

La validación cruzada es una técnica importante para evaluar el rendimiento de un modelo de manera más robusta:

```
from sklearn.model_selection import cross_val_score# Crear modeloknn
= KNeighborsClassifier(n_neighbors=3)# Realizar validación
cruzadacv_scores = cross_val_score(knn, X_scaled, y, cv=5)# Mostrar
resultadosprint(f"Puntuaciones de CV: {cv_scores}")print(f"Exactitud
media: {cv_scores.mean():.2f}")print(f"Desviación estándar:
{cv_scores.std():.2f}")
```



La validación cruzada divide los datos en k pliegues (folds) y entrena k modelos diferentes, cada uno utilizando k-1 pliegues para entrenamiento y 1 pliegue para validación.

alkemy

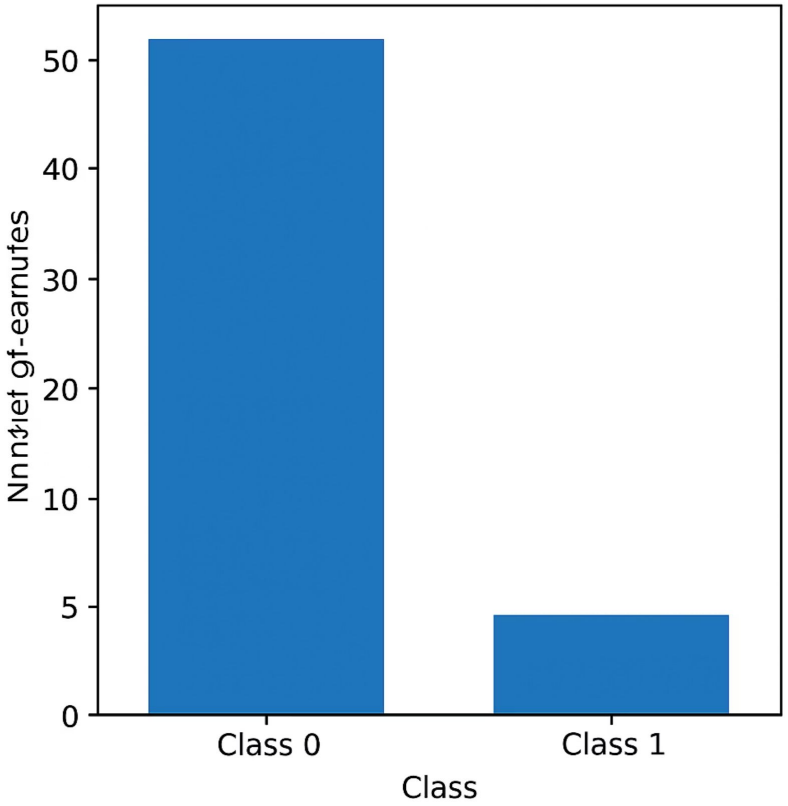
Manejo de Datos Desbalanceados

Cuando las clases no están equilibradas, la exactitud puede no ser una métrica adecuada. En estos casos, es importante considerar:

- Precisión: proporción de predicciones positivas correctas
- Sensibilidad (recall): proporción de positivos reales identificados
- Puntuación F1: media armónica de precisión y sensibilidad
- Área bajo la curva ROC (AUC-ROC)

También se pueden aplicar técnicas para equilibrar las clases:

- Sobremuestreo de la clase minoritaria (SMOTE)
- Submuestreo de la clase mayoritaria
- Ponderación de clases



alkemy

Aplicaciones Prácticas de K-NN

Diagnóstico Médico

Clasificación de pacientes según síntomas y resultados de pruebas para predecir enfermedades.

Análisis Financiero

Evaluación de riesgo crediticio y detección de fraudes basándose en patrones de transacciones.

Sistemas de Recomendación

Sugerencia de productos o contenido basándose en preferencias de usuarios similares.

Reconocimiento de Imágenes

Clasificación de imágenes según características visuales para identificación de objetos.

K-NN para Regresión

Aunque hemos enfocado el algoritmo K-NN para tareas de clasificación, también puede utilizarse para problemas de regresión:

- En lugar de asignar la clase más común, se calcula el promedio de los valores de los k vecinos más cercanos.
- Se puede aplicar una ponderación según la distancia para dar más importancia a los vecinos más cercanos.
- Es útil para predecir valores continuos como precios, temperaturas, etc.

Implementación en Scikit-Learn:

```
from sklearn.neighbors import KNeighborsRegressor# Crear y entrenar el modeloknn_reg = KNeighborsRegressor(n_neighbors=3, weights='distance')knn_reg.fit(X_train_scaled, y_train_reg)# Realizar prediccionesy_pred_reg = knn_reg.predict(X_test_scaled)# Evaluar el modelofrom sklearn.metrics import mean_squared_error, r2_scoremse = mean_squared_error(y_test_reg, y_pred_reg)r2 = r2_score(y_test_reg, y_pred_reg)print(f"Error cuadrático medio: {mse:.2f}")print(f"R²: {r2:.2f}")
```

alkemy

Comparación con Otros Algoritmos

K-NN vs. Árboles de Decisión

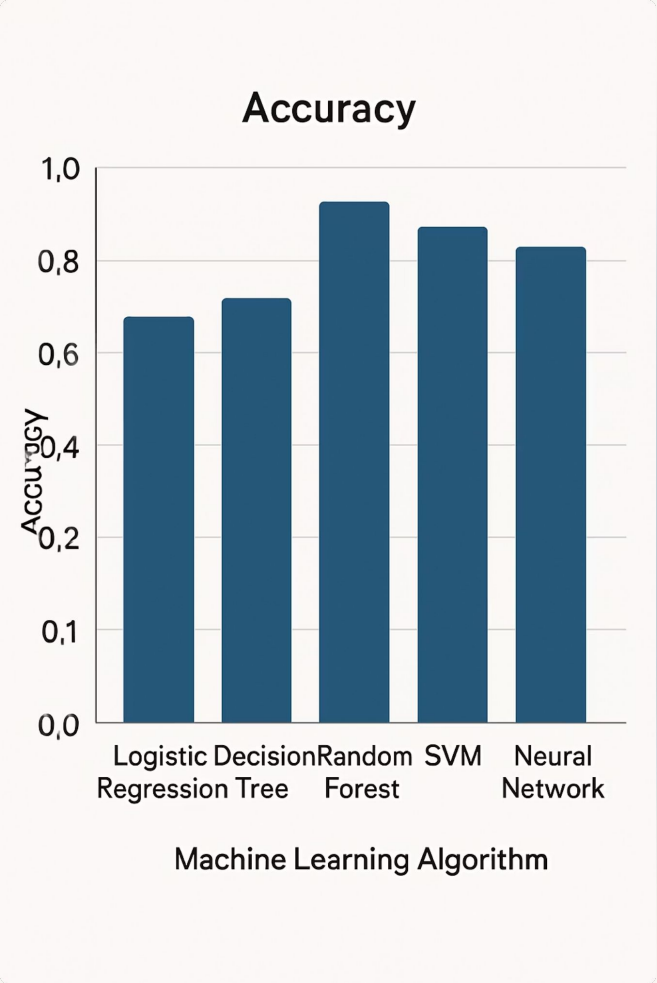
Los árboles de decisión crean reglas explícitas basadas en los datos, mientras que K-NN utiliza la similitud entre observaciones. Los árboles son más interpretables, pero K-NN puede capturar relaciones más complejas.

K-NN vs. Regresión Logística

La regresión logística asume una relación lineal entre variables y la probabilidad de pertenencia a una clase. K-NN no hace esta suposición, lo que lo hace más flexible para relaciones no lineales.

K-NN vs. SVM

Las máquinas de vectores de soporte (SVM) buscan un hiperplano óptimo para separar clases. K-NN es más simple y no requiere entrenamiento, pero SVM puede manejar mejor espacios de alta dimensionalidad.



Rendimiento Comparativo

El rendimiento de K-NN en comparación con otros algoritmos depende de la naturaleza de los datos y el problema específico. Es recomendable probar varios algoritmos y comparar sus resultados utilizando validación cruzada.

alkemy

Mejores Prácticas para K-NN

Preprocesamiento

Siempre escalar los datos antes de aplicar K-NN para evitar que variables con rangos mayores dominen el cálculo de distancias.

Selección de Características

Eliminar variables irrelevantes o redundantes para mejorar el rendimiento y reducir el tiempo de cálculo.

Optimización de K

Probar diferentes valores de K y seleccionar el que proporcione el mejor rendimiento en validación cruzada.

Evaluación Robusta

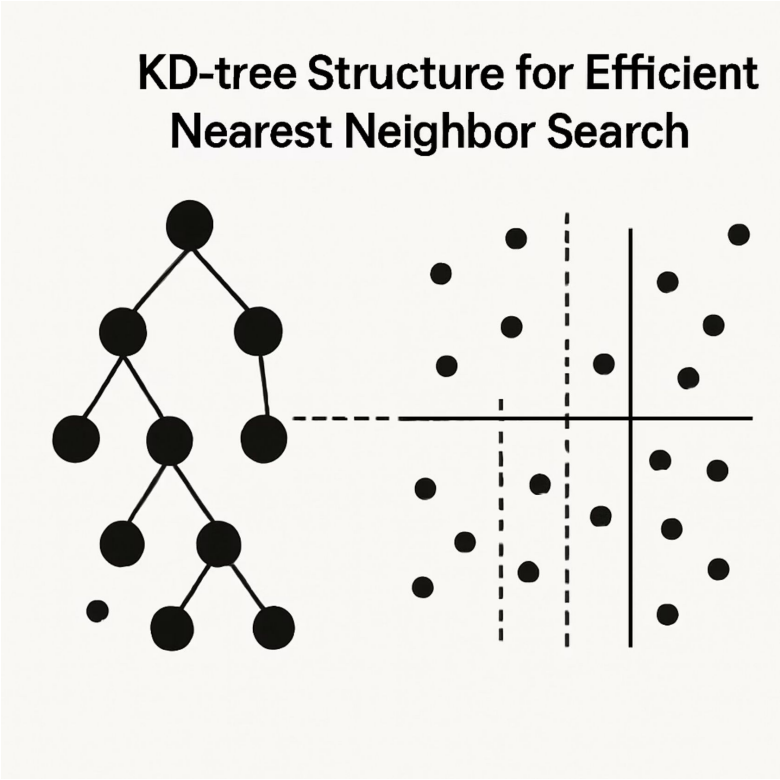
Utilizar múltiples métricas de evaluación y validación cruzada para obtener una estimación confiable del rendimiento.

Consideraciones para Conjuntos de Datos Grandes

K-NN puede ser computacionalmente costoso para conjuntos de datos grandes. Algunas estrategias para mejorar la eficiencia son:

- Utilizar estructuras de datos optimizadas como KD-Trees o Ball Trees
- Implementar búsqueda aproximada de vecinos más cercanos
- Reducir la dimensionalidad mediante técnicas como PCA
- Considerar algoritmos de aprendizaje basados en instancias más eficientes

Las estructuras de datos como KD-Trees organizan los puntos en el espacio para permitir búsquedas más eficientes de vecinos cercanos.



alkemy

Reducción de Dimensionalidad

La maldición de la dimensionalidad afecta especialmente a K-NN: en espacios de alta dimensionalidad, el concepto de "cercanía" pierde significado.

Técnicas de reducción de dimensionalidad que pueden ayudar:

- Análisis de Componentes Principales (PCA)
- t-SNE (t-Distributed Stochastic Neighbor Embedding)
- UMAP (Uniform Manifold Approximation and Projection)

```
from sklearn.decomposition import PCA# Aplicar PCApca =
PCA(n_components=2)X_pca = pca.fit_transform(X_scaled)# Entrenar
K-NN con datos reducidosknn =
KNeighborsClassifier(n_neighbors=3)knn.fit(X_pca, y)# Visualizar
resultadosplt.figure(figsize=(10, 8))plt.scatter(X_pca[:, 0],
X_pca[:, 1], c=[1 if y_i == 'Fútbol' else 0 for y_i in y],
cmap=ListedColormap(['#FF0000', '#0000FF']), edgecolor='k',
s=100)plt.title('Datos reducidos con PCA')plt.xlabel('Componente
Principal 1')plt.ylabel('Componente Principal 2')plt.show()
```

Manejo de Variables Categóricas

K-NN trabaja con distancias numéricas, por lo que las variables categóricas deben transformarse:

- One-Hot Encoding para variables nominales (sin orden)
- Codificación ordinal para variables con orden intrínseco
- Considerar métricas de distancia específicas para datos mixtos

```
from sklearn.preprocessing import OneHotEncoder# Datos
categóricosX_cat = np.array([['Rojo'], ['Azul'], ['Verde'],
['Rojo']])# Aplicar One-Hot Encodingencoder =
OneHotEncoder()X_encoded =
encoder.fit_transform(X_cat).toarray()print("Datos
originales:")print(X_cat)print("\nDatos
codificados:")print(X_encoded)
```

Salida:

```
Datos originales:[['Rojo'] ['Azul'] ['Verde']
['Rojo']]Datos codificados:[[0. 0. 1.] [1. 0. 0.] [0.
1. 0.] [0. 0. 1.]]
```

alkemy

Interpretación de Resultados

Matriz de Confusión

Muestra la cantidad de predicciones correctas e incorrectas para cada clase. Permite identificar en qué clases el modelo tiene mejor o peor rendimiento.

Informe de Clasificación

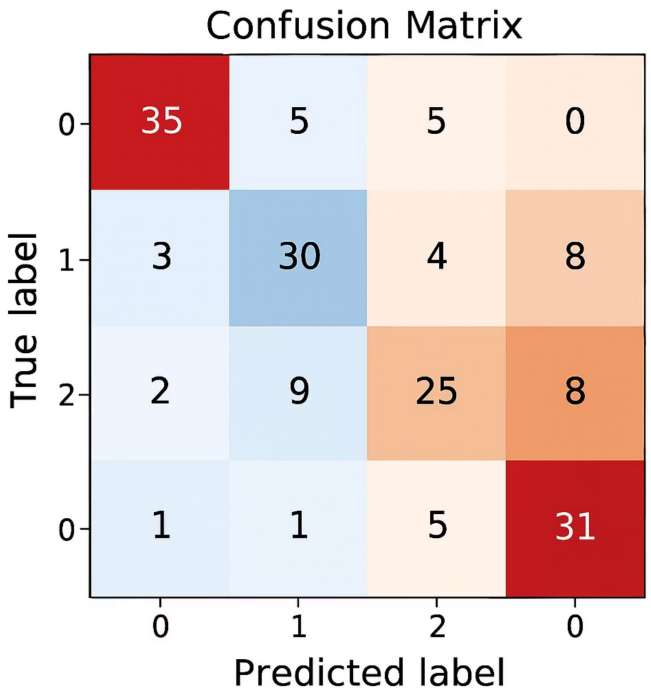
Proporciona métricas detalladas como precisión, sensibilidad y puntuación F1 para cada clase. Útil para evaluar el rendimiento en problemas con clases desbalanceadas.

Curva ROC y AUC

La curva ROC muestra la relación entre la tasa de verdaderos positivos y falsos positivos. El área bajo la curva (AUC) es una medida de la capacidad discriminativa del modelo.

Visualización de la Matriz de Confusión

```
import seaborn as snsimport matplotlib.pyplot as pltfrom
sklearn.metrics import confusion_matrix# Crear matriz de confusióncm
= confusion_matrix(y_test, y_pred)# Visualizarplt.figure(figsize=(8,
6))sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
xticklabels=['Baloncesto', 'Fútbol'],
yticklabels=['Baloncesto',
'Fútbol'])plt.xlabel('Predicción')plt.ylabel('Valor
Real')plt.title('Matriz de Confusión')plt.show()
```

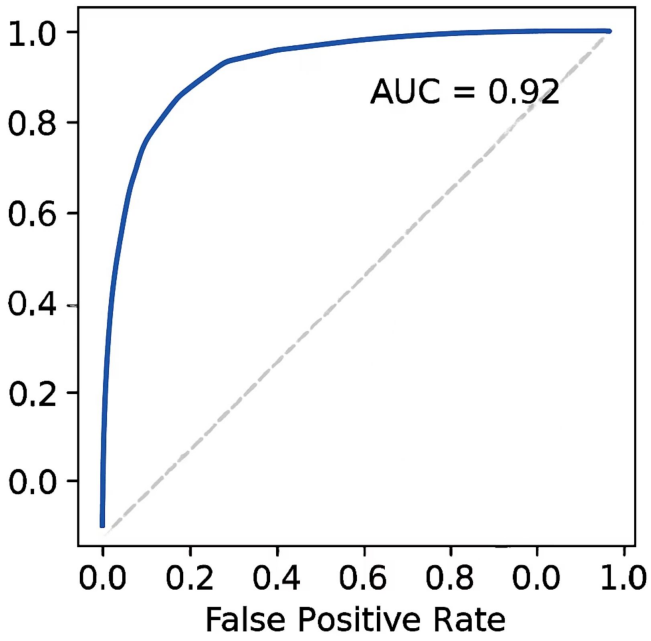


La matriz de confusión muestra cuántas observaciones de cada clase fueron clasificadas correctamente (diagonal) e incorrectamente (fuera de la diagonal).

alkemy

Curva ROC

```
from sklearn.metrics import roc_curve, aucimport matplotlib.pyplot as
plt# Obtener probabilidades de prediccióny_scores =
knn.predict_proba(X_test_scaled)[:, 1]# Calcular curva ROCfpr, tpr,
thresholds = roc_curve(    [1 if y == 'Fútbol' else 0 for y in
y_test],     y_scores)roc_auc = auc(fpr, tpr)#
Visualizarplt.figure(figsize=(8, 6))plt.plot(fpr, tpr,
color='darkorange', lw=2,          label=f'AUC =
{roc_auc:.2f}')plt.plot([0, 1], [0, 1], color='navy', lw=2,
linestyle='--')plt.xlim([0.0, 1.0])plt.ylim([0.0,
1.05])plt.xlabel('Tasa de Falsos Positivos')plt.ylabel('Tasa de
Verdaderos Positivos')plt.title('Curva ROC')plt.legend(loc="lower
right")plt.show()
```



La curva ROC es una representación gráfica de la capacidad discriminativa del modelo. Un modelo perfecto tendría un AUC de 1.0, mientras que un modelo aleatorio tendría un AUC de 0.5.

alkemy

Ajuste de Umbrales de Decisión

En problemas donde el costo de los falsos positivos y falsos negativos es diferente, puede ser necesario ajustar el umbral de decisión:

```
import numpy as npimport matplotlib.pyplot as pltfrom sklearn.metrics import precision_recall_curve# Obtener probabilidades de
prediccióny_scores = knn.predict_proba(X_test_scaled)[:, 1]# Calcular precisión y sensibilidad para diferentes
umbralesprecision, recall, thresholds = precision_recall_curve( [1 if y == 'Fútbol' else 0 for y in y_test],  y_scores)#
Visualizarplt.figure(figsize=(10, 6))plt.plot(thresholds, precision[:-1], 'b--', label='Precisión')plt.plot(thresholds,
recall[:-1], 'g-', label='Sensibilidad')plt.xlabel('Umbral de Decisión')plt.ylabel('Puntuación')plt.title('Precisión y
Sensibilidad vs. Umbral')plt.legend()plt.grid(True)plt.show()# Seleccionar umbral óptimo según criterio específico# Por ejemplo,
maximizar F1f1_scores = 2 * (precision[:-1] * recall[:-1]) / (precision[:-1] + recall[:-1])optimal_idx =
np.argmax(f1_scores)optimal_threshold = thresholds[optimal_idx]print(f"Umbral óptimo: {optimal_threshold:.2f}")
```

Análisis de Importancia de Características

A diferencia de otros algoritmos como árboles de decisión, K-NN no proporciona directamente la importancia de las características. Sin embargo, se pueden utilizar métodos indirectos:

- Eliminación recursiva de características
- Selección de características basada en correlación
- Análisis de componentes principales

```
from sklearn.feature_selection import RFECV# Crear modelo baseknn = KNeighborsClassifier(n_neighbors=3)# Aplicar eliminación recursiva de característicasselector = RFECV(knn, step=1, cv=5)selector.fit(X_scaled, y)# Mostrar características seleccionadasprint("Características seleccionadas:", selector.support_)print("Ranking de características:", selector.ranking_)print("Número óptimo de características:", selector.n_features_)
```

alkemy

Implementación en Producción

Serialización del Modelo

Guardar el modelo entrenado y los transformadores de datos

```
import joblib# Guardar modelo y
transformadoresjoblib.dump(scaler,
'scaler.pkl')joblib.dump(knn, 'knn_model.pkl')
```

Creación de Pipeline

Combinar preprocesamiento y modelo en un pipeline para

```
from sklearn.pipeline import Pipeline# Crear
pipelinepipeline = Pipeline([    ('scaler',
StandardScaler()),    ('knn',
KNeighborsClassifier(n_neighbors=3))])# Entrenar
pipelinepipeline.fit(X_train, y_train)# Guardar
pipeline completojoblib.dump(pipeline,
'knn_pipeline.pkl')
```

Despliegue

Implementar el modelo en un entorno de producción, como una API REST o un servicio web.

Monitoreo

Supervisar el rendimiento del modelo en producción y reentrenar cuando sea necesario.

alkemy

Ejemplo de Uso del Modelo Guardado

```
import joblibimport numpy as np# Cargar pipelinepipeline = joblib.load('knn_pipeline.pkl')# Datos nuevos para
predicciónnuevos_datos = np.array([[172, 72], [163, 62]])# Realizar predicciónpredicciones =
pipeline.predict(nuevos_datos)probabilidades = pipeline.predict_proba(nuevos_datos)print("Predicciones:",
predicciones)print("Probabilidades:", probabilidades)
```

Este enfoque garantiza que todos los pasos de preprocesamiento se apliquen correctamente a los nuevos datos antes de realizar la predicción.

Desafíos Comunes y Soluciones

1

Tiempo de Predicción Lento

- Utilizar estructuras de datos optimizadas (KD-Trees, Ball Trees)
- Reducir la dimensionalidad de los datos
- Implementar aproximaciones de vecinos más cercanos

2

Sensibilidad al Ruido

- Aplicar técnicas de detección y eliminación de valores atípicos
- Utilizar ponderación por distancia para reducir la influencia de puntos lejanos
- Implementar técnicas de selección de características

3

Maldición de la Dimensionalidad

- Reducir la dimensionalidad mediante PCA, t-SNE o UMAP
- Seleccionar características relevantes
- Considerar métricas de distancia adaptadas a espacios de alta dimensionalidad

4

Clases Desbalanceadas

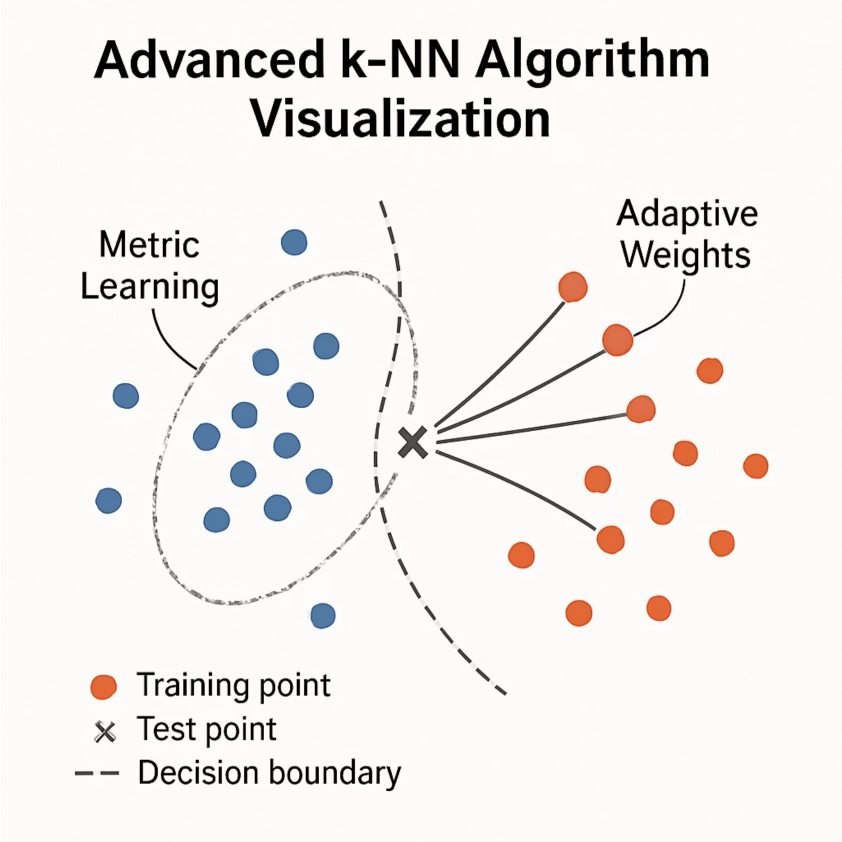
- Aplicar técnicas de sobremuestreo o submuestreo
- Utilizar métricas de evaluación adecuadas (F1, AUC)
- Ajustar los umbrales de decisión

Tendencias y Avances en K-NN

Aunque K-NN es un algoritmo clásico, continúa evolucionando con nuevas variantes y mejoras:

- K-NN con aprendizaje de métricas: adapta la función de distancia a los datos
- K-NN aproximado: algoritmos que encuentran vecinos cercanos de manera eficiente
- K-NN distribuido: implementaciones para conjuntos de datos muy grandes
- K-NN con pesos adaptativos: ajusta la influencia de cada vecino de forma dinámica

Estas variantes buscan superar las limitaciones tradicionales de K-NN, mejorando su eficiencia y precisión en diversos escenarios.



alkemy

Recursos Adicionales

Documentación Oficial

Scikit-Learn: Documentación de KNeighborsClassifier

Contiene explicaciones detalladas sobre los parámetros y métodos disponibles.

Libros Recomendados

"Introduction to Machine Learning with Python" de Andreas Müller y Sarah Guido

"Pattern Recognition and Machine Learning" de Christopher Bishop

Cursos en Línea

Coursera: "Machine Learning" de Andrew Ng

edX: "Introduction to Artificial Intelligence" de Microsoft

Comunidades

Stack Overflow: etiqueta [scikit-learn]

GitHub: repositorio scikit-learn para ejemplos y discusiones

Cierre

El algoritmo K-Nearest Neighbors (K-NN) es una de las técnicas de clasificación más intuitivas y utilizadas en el campo del aprendizaje automático. A través de este manual, hemos recorrido los conceptos fundamentales que sustentan su funcionamiento, como el cálculo de la distancia euclideana, la importancia del escalamiento de datos y la selección del valor óptimo de K.

Hemos visto que, si bien K-NN es un algoritmo fácil de implementar y comprender, también presenta desafíos relacionados con el tiempo de predicción, la escala de las variables y la selección de parámetros. Por ello, su correcta aplicación requiere un preprocesamiento cuidadoso y una evaluación adecuada utilizando métricas como la exactitud y la matriz de confusión.

Además, hemos implementado un flujo completo utilizando la librería Scikit-Learn, desde la preparación de los datos hasta la evaluación del modelo. Esto proporciona al lector una base práctica sólida para aplicar este algoritmo en problemas reales de clasificación.

Dominar K-NN no solo permite resolver problemas sencillos, sino que también ayuda a comprender principios fundamentales del aprendizaje automático supervisado, como la proximidad entre datos, la elección de métricas y la validación de modelos.

alkemy

Live Coding

¿En qué consistirá la Demo?

Vamos a construir paso a paso un modelo de clasificación K-NN utilizando un dataset de preferencias deportivas. Vamos a optimizar el valor de K, visualizar fronteras de decisión y evaluar el rendimiento usando métricas específicas.

1. *Cargar el dataset deportes.csv con altura, peso y deporte (target).*
2. *Escalar las variables numéricas (StandardScaler).*
3. *Dividir en train y test (70/30).*
4. *Entrenar un modelo base de KNeighborsClassifier con K=3.*
5. *Evaluar con accuracy_score, classification_report y matriz de confusión.*
6. *Probar distintos valores de K con GridSearchCV.*
7. *Visualizar cómo cambia el rendimiento con distintos K.*
8. *Graficar las fronteras de decisión en un gráfico 2D.*
9. *Guardar el modelo entrenado con joblib.*

Tiempo: 25 Minutos

Momento:

Time-out!

⏰5 -10 min.

alkemy

Ejercicio

Clasificador Deportivo K-NN

Clasificador Deportivo K-NN

Contexto: 🙌

Estás trabajando en un sistema que recomienda deportes según características físicas de los usuarios (altura y peso). Vas a construir un modelo de clasificación con K-NN que prediga si una persona practica fútbol o baloncesto, basado en un conjunto de datos etiquetado.

Consigna: ✍️

1. Cargue los datos de altura y peso junto con la etiqueta del deporte.
2. Escale las variables usando StandardScaler.
3. Divide los datos en entrenamiento y test (70/30).
4. Entrene un modelo K-NN (K=3) y evalue con matriz de confusión y accuracy.
5. Visualice las fronteras de decisión en un gráfico 2D.
6. Probá valores de K entre 1 y 10, evaluando con validación cruzada.
7. Exporte el modelo entrenado como .pkl.

Paso a paso: ⚙️

1. Importar librerías necesarias
2. Escalar y dividir los datos
3. Entrenar el modelo y hacer predicciones
4. Visualizar resultados y optimizar
5. Guardar modelo para producción

Tiempo 🕛**:** *45 Minutos*

alkemy

¿Alguna consulta?

alkemy

Resumen

¿Qué logramos en esta clase?

- ✓ ***Implementamos K-NN en un caso real.***
- ✓ ***Ajustamos parámetros y visualizamos decisiones.***
- ✓ ***Evaluamos con validación cruzada.***
- ✓ ***Exploramos su aplicación a regresión.***

¡Ponte a prueba!

Momento de ejercitación

Te invitamos a aprovechar esta última sección del espacio sincrónico para realizar de manera individual las ***actividades disponibles en la plataforma.*** *Estas propuestas son clave para afianzar lo trabajado y* ***forman parte obligatoria del recorrido de aprendizaje.***

💪 Análisis de caso ——— 💪 Selección Múltiple

💪 Comprensión lectora

Si al resolverlas surge alguna duda, compartela o tráela al próximo encuentro sincrónico.

alkemy

< ¡Muchas gracias! >

alkemy